

Mahjong Reinforcement Learning Agent

IEMS5726AB Data Science in Practice Project Report Draft

1. Problem Definition

This project develops a browser-playable Japanese mahjong demonstration in which a human player competes against two random baseline agents and one reinforcement-learning agent slot based on Mortal. The application aims to make an RL mahjong policy observable: users can inspect each player's hand/discards, follow the event log, and see the RL agent's discard reasoning and action-value scores.

2. Data Science Pipeline

- Data collection: Mortal supports mjai/Tenhou-style mahjong records, typically stored as compressed JSON log files.
- Preprocessing and representation: game events are converted into player observations and legal-action masks by Mortal's libriichi environment.
- Modeling: Mortal uses a residual neural network encoder and DQN action-value heads to select legal mahjong actions.
- Inference: the project includes a Python CLI adapter that can call Mortal's documented ``mortal.py`` interface when a trained checkpoint is configured.
- Fallback demonstration: because model weights should not be shipped inside the source archive, the browser demo uses a deterministic Mortal-inspired value policy when the checkpoint is unavailable.

3. System Architecture and UI/UX

Layer	Implementation	Purpose
Source model	Mortal-for-mahjong/Mortal	Training, observation encoding, DQN model, and CLI inference
Agent adapter	source_code/mahjong_agent/agent.py	Loads Mortal CLI when possible and falls back to an explainer
Application	application/index.html, styles.css, application.js	Playable mahjong demo with human, random agents, and RL agent
Submission metadata	application/model_link.txt	External checkpoint link placeholder required by course instructions

The UI is designed for the short demonstration video: the first screen is the actual mahjong table, with controls for starting a new round and advancing agents. The side panel explains the latest RL decision through ranked action values instead of hiding the model behind a black box.

4. Challenges

- Mortal requires trained checkpoints and compiled dependencies; the project separates source code from model weights and documents how to connect the external checkpoint.
- A full mahjong engine is complex, so the demo focuses on the course-required human-vs-agent interaction loop and makes draw/discard decisions clear.
- To keep the demo reliable for grading, the RL slot has an explainable deterministic fallback when the model file is unavailable.
- The interface must be understandable in a short video, so the page emphasizes visible table state, event history, and action-value visualization.

5. References

- Equim-chan, Mortal: <https://github.com/Equim-chan/Mortal>
- Mortal documentation and mjai inference example: <https://mortal.ekyu.moe>
- ReportLab for generating this PDF draft: <https://www.reportlab.com>

6. VeriGuide Receipt

Attach the signed VeriGuide receipt before final submission. Only one group member needs to submit the report to VeriGuide according to the project instructions.